
Personalized News Aggregator

Proof-of-Concept Build Plan & Technical Roadmap

Document type:	Technical build plan / consulting scope
Prepared for:	Consulting engagement — POC evaluation
Core concept:	Aggregated + personalized news feed with owner-published content
Monetization path:	AdSense (Phase 1) → Subscriptions (Phase 2)

This document outlines the architecture, data model, legal considerations, phased build sequence, and estimated timeline/cost for a proof-of-concept personalized news aggregation platform, intended as a scoping reference ahead of a build engagement.

1. Executive Summary

The goal of this POC is to validate a news aggregation platform that pulls headlines, thumbnails, and short snippets from external news sources, displays them in a scrollable feed, and links out to the original article on click. On top of this base aggregation layer, the platform adds two differentiators: (1) per-user personalization based on selected topics and sources, and (2) an owner/editor publishing tool for original content. Initial monetization is via Google AdSense, with a subscription tier planned as a Phase 2 addition once the product has traction.

This model mirrors proven aggregator products (Google News, SmartNews, Flipboard) and is technically well understood — the main risks are legal/licensing (how much content can be displayed from a third-party source) and content-source reliability (RSS feed uptime and formatting inconsistency across publishers), not algorithmic complexity.

Area	Summary
Core function	Aggregate headline + image/video + snippet from external RSS/APIs; click-through to source
Personalization	User selects preferred topics/sources at signup; feed filters accordingly
Owner content	Admin publishing tool for original articles, featured in feed
Monetization v1	Google AdSense display ads within feed and sidebar
Monetization v2	Stripe subscription gating ad-free / exclusive content
Estimated POC timeline	6–8 weeks, part-time solo build (see Section 6)

2. System Architecture

The system is composed of four layers: a content ingestion pipeline, a data store, a personalization/query layer, and a front-end feed renderer. A background worker process handles scheduled pulls from external sources so the user-facing app never waits on a live scrape.

Layer	Responsibility	Suggested Tech
Ingestion worker	Poll RSS feeds / news API on a schedule (every 15–30 min); parse and normalize into a common article schema; dedupe by URL hash	Node.js cron job or Vercel Cron + a queue (BullMQ/Upstash Redis)
Data store	Persist articles, users, preferences, admin posts, click/engagement events	PostgreSQL via Supabase or Railway
API / app layer	Serve personalized feed queries, handle auth, expose admin CMS endpoints	Next.js (API routes) + Prisma ORM
Auth	User signup/login, session management, admin role flag	Clerk
Front end	Card-based feed UI, infinite scroll, profile/preferences screen, admin post form	Next.js + React + Tailwind

Layer	Responsibility	Suggested Tech
Media handling	Store/serve thumbnails for owner-posted content (third-party images are hotlinked/cached only with permission)	Cloudflare R2
Ads	AdSense units injected into feed at fixed intervals	Google AdSense script

2.1 Data Flow (Ingestion → Feed)

- Scheduled worker fetches configured RSS feeds / API sources.
- Parser normalizes each item into: title, image URL, snippet, source name, source URL, category, published_at.
- New items are deduplicated (by canonical URL) and inserted into the **Articles** table.
- When a user requests their feed, the API filters Articles by the user's selected topics/sources, merges in any Owner-posted Articles flagged for their categories, sorts by recency, and paginates.
- Clicking a card logs a click event (for future recommendation tuning) and redirects to the original source URL in a new tab — the article body is never rendered on-platform.

3. Data Model (Core Tables)

3.1 Article

Field	Type	Notes
id	UUID (PK)	Primary key
title	text	Headline as pulled from source
snippet	text	Short excerpt, 1–2 sentences max
image_url	text	Thumbnail/video preview URL (hotlinked or cached with permission)
media_type	enum	image / video
source_name	text	e.g. "Reuters", "AP"
source_url	text	Canonical link to original article (click-through target)
category	text[]	e.g. ["politics", "technology"]
published_at	timestamp	Original publish time
ingested_at	timestamp	When our worker pulled it
is_owner_content	boolean	True for owner/editor-published originals

3.2 User & UserPreferences

Field	Type	Notes
id	UUID (PK)	Managed via Clerk, mirrored locally

Field	Type	Notes
email	text	
display_name	text	
is_admin	boolean	Gates access to owner publishing tool
created_at	timestamp	
— UserPreferences —		
user_id	UUID (FK)	1:1 with User
preferred_topics	text[]	e.g. ["business", "sports"]
preferred_sources	text[]	e.g. ["BBC", "AP"]
updated_at	timestamp	

3.3 Source & ClickEvent

Field	Type	Notes
— Source —		
id	UUID (PK)	
name	text	e.g. "NPR"
feed_url	text	RSS endpoint or API source ID
categories	text[]	Topics this source is tagged under
is_active	boolean	Toggle off broken/unreliable feeds
— ClickEvent —		
id	UUID (PK)	
user_id	UUID (FK, nullable)	Null for anonymous clicks
article_id	UUID (FK)	
clicked_at	timestamp	

4. Legal & Content Considerations

This is the area with the most real risk in the product, more so than any technical build risk. The safest and most standard model — the one used by Google News, SmartNews, and similar aggregators — is:

- Display only headline, thumbnail image, and a short snippet (1–2 sentences) pulled from the source's own RSS feed metadata — never the full article body.
- Always link out to the original source for the full article; do not reproduce or reformat the article content on your own domain.
- Attribute the source clearly on every card (publisher name/logo).
- Prefer sources that explicitly publish RSS feeds for syndication, or use a licensed aggregation API (NewsAPI, GNews, Currents), which handles source licensing on your behalf.
- Avoid scraping full page content directly from sites that don't offer a feed/API — this is where legal exposure increases significantly.
- Keep a documented takedown process: if a publisher requests removal, pull their feed immediately.

None of this constitutes legal advice — before launch, a short consult with an attorney familiar with digital media/IP (Foundry Law Group could potentially advise here) is worth the cost to confirm the specific sources and API terms you plan to use.

5. Recommended Tech Stack

Component	Choice	Why
Framework	Next.js (React)	Single framework for front end + API routes; strong Vercel deploy story
Database	PostgreSQL (Supabase)	Relational fit for structured articles/users; generous free tier
ORM	Prisma	Type-safe schema/migrations
Auth	Clerk	Fast to integrate, handles sessions/roles out of the box
Background jobs	BullMQ + Upstash Redis (on Railway)	Reliable scheduled RSS polling outside of request/response cycle
Media storage	Cloudflare R2	Cheap object storage for owner-uploaded images
Hosting	Vercel (app) + Railway (worker)	Low-ops, scales down to near-zero cost at POC stage
Ads	Google AdSense	Fastest path to Phase 1 revenue, no sales process needed
Payments (Phase 2)	Stripe	Standard subscription billing, minimal custom code

This stack overlaps closely with tooling already in use on other projects, which should reduce ramp-up time considerably.

6. Phased Build Plan

The plan below sequences the build so that a demoable product exists as early as possible (end of Phase 1), with each subsequent phase adding a discrete, testable capability.

Phase 0: Setup & Source Selection · Est. 3–4 days

Goal: Stand up the project skeleton and lock in initial content sources.

- Initialize Next.js project, Postgres DB (Supabase), Prisma schema for Article/Source tables.
- Identify and vet 10–15 RSS feeds across target categories (general, business, tech, sports, etc.); confirm each publisher's feed is intended for syndication.
- Set up repo, environment configs, and deployment pipeline (Vercel).

Deliverable: Working repo with schema deployed and a vetted source list.

Phase 1: Core Aggregation Feed (Public, No Auth) · Est. 1.5–2 weeks

Goal: Prove the base aggregator concept end-to-end.

- Build ingestion worker: pull RSS on a schedule, parse into normalized Article records, dedupe.
- Build public feed UI: card grid/list with image or video thumbnail, headline, source attribution.
- Click on a card opens the source URL in a new tab (no article body rendered locally).
- Basic category filter tabs (e.g. Top, Business, Sports, Tech) — not yet personalized per user.

Deliverable: A live, publicly browsable aggregated news feed — this alone is demoable to the friend/stakeholder.

Phase 2: User Accounts & Personalization · Est. 1–1.5 weeks

Goal: Let each user get a feed tailored to their interests.

- Integrate Clerk for signup/login.
- Build onboarding step: select preferred topics and/or sources.
- Store selections in UserPreferences; modify feed query to filter by these.
- Add a settings page so a user can update preferences after signup.

Deliverable: Logged-in users see a feed scoped to their chosen topics/sources; anonymous visitors still see the general feed.

Phase 3: Owner Publishing Tool · Est. 4–6 days

Goal: Give the site owner a way to publish original content alongside aggregated content.

- Add `is_admin` flag and a protected `/admin` route.
- Build a simple post form: title, category, body text, image upload (to Cloudflare R2).
- Owner-posted articles render as full on-site pages (unlike aggregated cards, which link out) and are tagged "Original" in the feed.
- Feature these prominently (e.g. pinned row at top of feed).

Deliverable: Owner can publish and manage original articles that appear inline with aggregated content.

Phase 4: Monetization — AdSense · Est. 2–3 days

Goal: Turn on the first revenue stream.

- Apply for / configure Google AdSense on the domain.
- Insert ad units at fixed intervals in the feed (e.g. every 5th card) and in a sidebar slot.
- Confirm layout doesn't violate AdSense placement policies (no accidental clicks, clear ad labeling).

Deliverable: Live ad units generating impressions/revenue.

Phase 5: Polish, QA & Soft Launch · Est. 1 week

Goal: Stabilize for a small real-user test group.

- Mobile responsiveness pass on feed and admin views.
- Error handling for broken/slow RSS sources (skip and log rather than fail the whole ingestion run).
- Basic analytics (page views, click-through rate per source/category) to inform Phase 2 personalization.
- Invite a small group of test users; gather feedback on relevance of personalized feed.

Deliverable: POC ready to demo/pitch, with real usage data starting to accumulate.

Estimated total: ~6–8 weeks for a single developer working part-time (evenings/naptime blocks), assuming no major surprises in source reliability or auth integration.

7. Phase 2 — Post-POC Roadmap (Not in Initial Build)

- **Subscriptions:** Stripe integration gating an ad-free experience and/or exclusive owner content behind a paid tier.
- **Smarter personalization:** Use click/engagement history to refine ranking beyond simple topic/source filters (collaborative filtering or a lightweight ML ranking model).
- **Push/email digests:** Daily or weekly personalized digest emails to drive return visits.
- **Licensed content API:** Move from raw RSS to a paid aggregation API (e.g. NewsAPI) for broader, more reliable source coverage as the product scales.
- **Native mobile app:** Once web traction is proven, a wrapped mobile experience for push notifications and stickier engagement.

These are intentionally excluded from the POC scope to keep the initial build lean and focused on validating the core concept before adding cost or complexity.

8. Key Risks & Mitigations

Risk	Mitigation
Publisher objects to content use / licensing dispute	Headline+snippet+link-out only model; documented takedown process; legal review of source terms before launch

Risk	Mitigation
RSS feeds go down or change format unexpectedly	Ingestion worker logs failures per-source and skips rather than crashes; weekly manual check of source health
AdSense account rejection or policy violation	Follow placement policies closely; avoid excessive ad density; keep content original-owner posts policy-compliant
Thin content / low article volume at launch	Start with a broader source list (15+ feeds) across popular categories to ensure feed never feels empty
Personalization feels irrelevant with limited data	Keep v1 personalization simple (explicit topic/source selection) rather than promising smart recommendations too early